TALEEMABAD UNIVERSITY · AGENTIC AI MASTERY

Module 14 — Connecting Your Agent to Notion and Slack

Eight video scripts · six concept + two screencast walkthroughs · draft for review

One protagonist (Ali) · one setting (the back room of his stationery shop) · one running project (his order agent) · ~26 beats and ~2 minutes per video.
Videos 7 and 8 are step-by-step walkthroughs on the real Notion and Slack screens, with the capture list for every screen.
Every fact in these scripts traces to the sourced research brief at the back.
No art, voiceover or render has been generated. Nothing is built until you approve the scripts.

Contents

Module 14 — Connecting Your Agent to Notion and Slack

Series: 8 videos — **6 concept videos** (~2 min each, the illustrated Ali format) plus **2 screencast walkthroughs** (~2.5 min each, real Notion and Slack interfaces, click by click) · one protagonist (Ali) · one setting (the back room of his stationery shop) · one running project (his order agent).

Module promise: by the end, a beginner can connect their own agentic project to Notion and Slack without hitting the four errors that stop everyone, and knows which of the two doors — connector or token — their project actually needs.

Why this module exists

Beginners do not fail at this because the code is hard. They fail on five specific things, in this order:

1. They never decide what should cross the boundary, so they wire everything and own nothing. 2. They reach for raw API code for a job the built-in connector would have done in one command. 3. They create the integration and assume it can see the workspace. It cannot. `object_not_found`. 4. They copy an ID out of the browser URL and query the wrong kind of object. 5. They do the work before acknowledging Slack, and the same action fires three times.

Each video owns exactly one of those, stages the real error on screen, and hands over the single fix.

The videos

Concept videos 1–6 — why, and what goes wrong. Illustrated Ali format.

#	Title	Teaches (one concept)	The one move	The error it kills
1	**Locked Out of the Room**	An agent becomes useful when it can read where the team's memory lives and write where the team's attention is	Name one thing to read, one thing to write — one door each side	The great agent nobody uses
2	**Two Doors In**	Hosted MCP connector vs API token, and how to choose	Ask "am I at the keyboard, or is it running without me?"	Weeks lost hand-coding what one command does
3	**The Key Only Opens One Room**	Tokens and scopes are a hotel key card — and Notion makes you hand the card to the room	Share the page, add the scope, reinstall, invite the bot	`object_not_found` · `not_in_channel` · `missing_scope`
4	**Rows, Not Paragraphs**	Write structured properties into a data source, not prose onto a page	Discover the `data_source_id` from the database, then query that	Wrong-ID 400s and unfilterable notes
5	**Speaking Without Spamming**	Acknowledge in three seconds, work after, dedupe, reply in thread	`ack()` first, then do the job, keyed on `event_id`	Triple-posting and self-reply loops

#	Title	Teaches (one concept)	The one move	The error it kills
6	**Safe, Then Let It Run**	Secrets in the environment, a human before the irreversible action, a log of everything	Read-only in a test channel today, one write tomorrow	Revoked tokens and unrecoverable actions

Walkthroughs 7–8 — the actual click path, on the actual screens. Screencast format: `card` + `ui` beats on real Notion and Slack interfaces, no illustrated art, TTS-only cost (the same format as the Module 11 assessment guide, rendered by an `animation/walkthrough.html` built on `assessment.html`).

#	Title	Screens	The one move	What the viewer has at the end
7	**Notion, Click by Click**	10 screens: new integration → capabilities → secret → `.env` → the three-dot menu → Connections → confirm → parent page → the two ids in the address bar → first rows	Create it, share the page under Connections, discover the `data_source_id`	Their own rows printing in their own terminal
8	**Slack, Click by Click**	15 screens: create app → From scratch → admin notice → Bot Token Scopes → reinstall banner → Install to Workspace → `xoxb-` token → `.env` → test channel → `/invite` → joined → first message with the APP badge → threaded → Socket Mode	Add the scope, reinstall, copy the token, invite the bot	A threaded message from their own bot in their own test channel

Both walkthroughs turn on the same sentence — **permission, then membership** — which is the frame that makes them a pair instead of one long video. Video 8, beat 17 is that frame.

Watch order. 1 → 2 → 3 → **7** → 4 → 5 → **8** → 6. Concepts 3 and 4 earn the ideas that walkthrough 7 executes; 5 earns what 8 executes; 6 closes on safety once the thing actually runs. The videos are also built to stand alone, so a learner who only wants the click path can watch 7 and 8 back to back.

Videos 3, 4, 5, 7 and 8 are the ones learners will come back to — 7 and 8 will be the most-replayed in the whole module, so they carry a step counter (`3 / 7`) and hold the two pause-worthy frames longer than the voiceover needs.

What a learner needs BEFORE video 3

Lives on the module page and in the video descriptions. Only the admin-approval line is spoken on camera (video 2, beat 14) — the rest would cost story time for no teaching gain.

- A Notion workspace they can create an integration in, and one page they are willing to share with it.
- A Slack workspace where they are allowed to install an app — **or an admin who will approve it**. The official Slack MCP server requires workspace-admin approval.
- Node or Python installed, an editor, and a `.env` file that is already in `.gitignore`.
- A **test** Slack channel and a **test** Notion page. Not `#general`. Not the real order table.

Series continuity rules (locked)

- **Protagonist:** Ali, the same shopkeeper from the evals and autonomy series. Same face, same outfit, clean-shaven, one skin tone, across all six videos. Never renamed.

- **Setting const** (repeated verbatim in every `scene` art prompt): *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.*
- **The AI is the laptop.** A real laptop with a plain blank screen; its output is crisp on-screen HTML. Never a glowing orb, never a robot, never a floating brain.
- **Three staff** appear as illustrated figures in the shop, never named, never speaking lines.
- **No text baked into generated scene art.** Every error string, scope name and URL lives on an `info` card so it can be proofread.
- **Every video ends on one action the viewer can take today**, spoken, never a homework file.
- **The two walkthroughs are the one exception to the visual standard.** No illustrated setting, no real-laptop art, no `scene` quota, no Imagen spend and no `animateIds` — motion comes from the renderer (cursor travel, click pulses, typed text). Ali still opens and closes each one in the illustrated shop frame so the module reads as one series. Do not run `qa-visuals.js` against videos 7 and 8.
- **Nothing real on screen in the walkthroughs.** Invented shop data, invented channel names, masked tokens (masked in the render, never blurred afterwards), plain text wordmarks rather than vendor logos.

Module close

Video 6 closes the module on a go-live checklist the learner can run on their own project. The module does not end on "now you know how APIs work" — it ends on *your agent is reading one real table and posting one real line into one real channel, and you can undo everything it does.*

Video 1 — Locked Out of the Room — draft

Teaches: an agentic project only becomes useful when it can read where the team's memory lives and write where the team's attention is — so you decide what crosses that boundary before you touch a single settings page.

One move: name one thing your agent should read, and one thing it should write. One door each side.

Real project / everyday spine: Ali's order agent writes perfect summaries into a terminal window that nobody in his shop ever opens.

This video does not teach any setup. No tokens, no scopes, no endpoints — those are videos 2 and 3. Its whole job is to break the beginner instinct of "connect everything" and replace it with two named doors. The words **read** and **write**, and the pairing *Notion = memory / Slack = attention*, are planted here and reused in all five later videos.

Deferred: connector vs token (video 2), permissions and the errors (video 3), how to write a row (video 4), how to post without spamming (video 5), secrets and approval (video 6).

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	Your agent is only useful where your team already works.
2	SCENE	Ali runs a stationery shop with three staff and one laptop in the back.
3	SCENE	Every evening his agent reads the day's orders and writes a clean summary.
4	SCENE	The summary lands in a black terminal window on his laptop. *(act: the screen fills, nobody is looking at it)*
5	INFO	Twelve summaries this month. Nobody but Ali opened one. *(number card: 12 written · 1 reader)*
6	ALI	If you have built something good that nobody uses, this is why.
7	ALI	His staff keep asking the same questions in the shop group chat.
8	SCENE	And the answers sit in a fat paper file on the corner of his desk. *(act: a hand flipping pages)*
9	INFO	The agent cannot see the file, and cannot speak in the chat.
10	INFO	The team's memory lives in one place. Their attention lives in another. *(two-panel card)*
11	INFO	Notion is memory. It stays, and you can search it. *(card, left panel filling in)*
12	INFO	Slack is attention. It is read now, then it scrolls away. *(same card, right panel filling in)*
13	SCENE	Ali types the paper file into a Notion table on the laptop screen. *(act: crisp table appears on screen)*
14	ALI	Now his agent can read the exact records his staff read.
15	INFO	Reading and writing, across two tools, is four doors. *(2x2 grid card: read/write x Notion/Slack)*
16	ALI	Beginners open all four at once, and then trust none of them.

17	INFO	QUESTION · Your agent has just made a decision the team will need again in six months. Where should it put it? A — post it in the Slack channel · B — write it as a row in the Notion table · C — leave it in the terminal log · D — keep it in the agent's memory. Write your answer down. *(quiz card, no answer shown, holdAfter)*
18	ALI	Ali opens one door first, and only one.
19	ALI	The agent may read the orders table. Nothing else, not yet.
20	SCENE	A staff member asks about last Tuesday, and the answer arrives in the chat. *(act: Ali looks up from his mug, relieved)*
21	INFO	REVEAL · B, the Notion row. Slack scrolls away; Notion is still findable in six months. *(same quiz card, B highlighted)*
22	ALI	Nothing about his agent changed today. Only what it could reach.
23	INFO	One thing to read. One thing to write. Start there. *(closing card)*
24	ALI	Your turn. Name the one question your team asks every single week.
25	ALI	That question tells you which door to open first.

Gate check

READY · 25 beats · 8 [scene] (32%) · 7 [ali] · 10 [info] · average ~11 words per line

- **One move:** name one read and one write. New concepts across the whole script: read vs write, Notion-as-memory, Slack-as-attention, four doors, open one first. Five — under the eight ceiling.
- **Leads with the answer:** beat 1 states the whole thesis before any story.
- **Everyday picture:** a paper file on a desk and a group chat. Demonstrated (beats 7–8, 13), never defined. A beginner with no vocabulary can restate it.
- **Before → after:** before is twelve unread summaries in a terminal (4–5); after is a staff question answered in the chat from the Notion table (20). Both shown in the shop, both countable.
- **Character:** Ali built something good, watched it go unused, felt that, moved the file, opened one door, got the payoff. He travels; he never recites a definition.
- **Emotional checkpoints:** beat 6 normalises ("if you have built something good that nobody uses"), beat 16 forgives the over-connecting instinct, beat 22 reassures that no rebuild was needed. Three.
- **Interactive pair:** QUESTION at beat 17 (~⅔ through, `holdAfter: 6`), REVEAL at beat 21.
- **Numbers on screen:** 12 summaries / 1 reader (5), four doors (15). Countable stakes, no invented statistics.
- **Data templates:** number card (5), two-panel memory/attention card (10–12), 2x2 door grid (15), quiz card (17, 21). Four distinct templates. Plain statement cards: 9 and 23 — two, at the ceiling.
- **No assignments.** Beat 24 is a spoken reflection prompt.

Build notes

- `animateIds = ['04', '13', '20']` — beat 4 (the summary filling a terminal nobody watches), beat 13 (the paper file becoming a crisp table on the laptop screen — the visual hinge of the video), beat 20 (the answer arriving in the chat while Ali looks up). Motion teaches in all three; every other beat is a still with a slow pan plus cutout micro-motion on Ali.

- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.*
- **This video sets the module's look.** Beat 2 is the palette pilot; beats 2, 8, 13, 20 lock the desk, the file and the laptop angle for all six videos.
- Beat 4: show the actual bad outcome — a wall of terminal text, Ali's chair empty, the three staff visible through the doorway looking at their phones. Not "the summary was unused" in the abstract.
- Beats 10–12 must be **one evolving card**, not three cards: the frame appears empty, the memory panel fills, then the attention panel fills. Same geometry all three beats.
- Beat 15 is the door grid and it recurs in videos 2 and 6 — give it a strong, memorable composition and reuse the identical art there.
- Beats 17 and 21 are the same quiz card art; 21 only adds the highlight on B.
- Leave air after beat 9 (the locked-out landing) and after beat 22.
- **No text in any generated scene art.** "Notion", "Slack" and every label live only on `info` cards.

Video 2 — Two Doors In: Connector or Key — draft

Teaches: there are exactly two ways to connect an agentic project to Notion or Slack — a hosted MCP connector, or an API token — and which one you need is decided by one question, not by taste.

One move: ask "am I at the keyboard, or is it running without me?" — at the keyboard means connector, running without you means token.

Real project / everyday spine: Ali wants two things this week: to question his order records himself in the evening, and a digest waiting in the shop chat at six in the morning.

This video does not teach permissions or code. It only has to stop the beginner from hand-writing API calls for a job the built-in connector does in one command — the most expensive wrong turn in the module — and to plant the sentence that decides it. The two named servers are given exactly, because a beginner cannot guess a URL.

Deferred: tokens, scopes and the errors (video 3), writing structured rows (video 4), events and acknowledgement (video 5), secrets and approval (video 6). Building your own MCP server is out of scope for the whole module.

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	If you are at the keyboard, use a connector. If it runs without you, use a token.
2	SCENE	Ali wants two things from his agent this week.
3	SCENE	In the evening, he wants to ask his order records questions himself.
4	SCENE	At six in the morning, he wants a digest already waiting in the shop chat.
5	INFO	Same two tools. Two completely different doors. *(two-job table card: evening / six a.m.)*
6	ALI	He starts where most beginners start, writing his own code for both.
7	SCENE	Three evenings later, he still has not asked a single question. *(act: notebook of half-written code, cold mug, clock late)*
8	ALI	That was not a coding failure. He picked the harder door for the easier job.
9	INFO	Door one is a connector, already built and kept up to date for you.
10	INFO	One line adds it. *(command card: claude mcp add --transport http notion https://mcp.notion.com/mcp)*
11	ALI	He signs in through the browser once, and there is no token to keep anywhere.
12	SCENE	That same evening he asks his records a question and reads the answer aloud. *(act: crisp answer on the laptop screen, Ali leaning in)*
13	INFO	Slack has an official one too. *(card: mcp.slack.com/mcp · generally available 17 February 2026)*
14	INFO	A workspace admin must approve it, so ask on day one, not on day three.
15	ALI	But the six o'clock digest has a problem the connector cannot solve.
16	SCENE	At six in the morning the back room is dark and the chair is empty. *(act: clock at 6, nobody there)*
17	INFO	A connector needs a person signed in. A token does not. *(contrast card — the hinge frame)*

18	INFO	QUESTION · Which of these needs an API token, not a connector? A — asking your notes a question while you work · B — posting a digest at six a.m. with nobody watching · C — drafting a page you will review right now · D — searching the workspace during a call. Write your answer down. *(quiz card, no answer shown, holdAfter)*
19	ALI	So Ali keeps the connector for himself, and adds a token for the schedule.
20	INFO	Tokens are also the only door for reacting the moment something changes. *(list card: runs on a schedule · reacts to an event · runs on a server)*
21	ALI	A connector waits to be asked. It cannot be woken up by a change.
22	INFO	REVEAL · B, because at six in the morning nobody is there to sign in. *(same quiz card, B highlighted)*
23	SCENE	One project, two doors, chosen job by job. *(act: Ali writes two lines on the back-room whiteboard)*
24	INFO	Ask one thing. Am I at the keyboard, or is it running without me? *(closing card, spoken in full)*
25	ALI	Your turn. Name the one job of yours that runs while you sleep.
26	ALI	That job needs a token. Everything else can start tonight with a connector.

Gate check

READY · 26 beats · 8 [scene] (31%) · 9 [ali] · 9 [info] · average ~12 words per line

- **One move:** apply the keyboard-or-not question. New concepts: connector, token, one-line install, admin approval, needs-a-person, schedules-and-events. Six — under the eight ceiling.
- **Leads with the answer:** beat 1 is the rule, before the story.
- **Everyday picture:** two jobs, one done sitting at the desk in the evening, one that has to happen while he is asleep. Concrete, and it *is* the technical distinction.
- **Before → after:** before is three evenings of half-written code and zero questions asked (6–8); after is a question answered the same evening (12) and a token added only where it earns its place (19). Both in the shop, both countable.
- **Character:** Ali picks the hard door, loses three evenings, is told plainly it was not a coding failure, switches, wins the evening back, then meets the one job the connector genuinely cannot do.
- **Emotional checkpoints:** beat 6 normalises the wrong turn, beat 8 explicitly removes the shame ("not a coding failure"), beat 21 reassures that the connector is not defective, just asleep. Three.
- **Interactive pair:** QUESTION at beat 18 (~⅔ through, `holdAfter: 6`), REVEAL at beat 22.
- **Numbers / names on screen:** three evenings lost (7), one line (10), `mcp.notion.com/mcp` (10), `mcp.slack.com/mcp` and 17 February 2026 (13), six a.m. (4, 16, 22). All verified in `research.md` (F1–F3).
- **Data templates:** two-job table (5), command card (10), named-server card (13), contrast card (17), list card (20), quiz card (18, 22). Six distinct. Plain statement cards: 9 and 14 — two, at the ceiling.
- **No assignments.** Beat 25 is a spoken reflection prompt.

Build notes

- `animateIds = ['07', '12', '16']` — beat 7 (three lost evenings: the cold mug, the clock, the half-written page), beat 12 (the answer landing on screen and Ali leaning in — the emotional turn of the video), beat 16 (the dark empty back room at six a.m., the beat the whole second half rests on).

- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.*
- Beat 16 is the only night-lit frame in the module: same room, same angle as beat 12, lights off, chair empty, clock readable. It must be recognisably the *same* desk — that is the whole point.
- **Beat 17 is the hinge frame.** "A connector needs a person signed in. A token does not." Clean two-column card, held. This sentence is quoted again in video 6.
- Beat 10 and 13 are **command / URL cards — proofread character by character** against `research.md` section D. A mistyped URL in a beginner video is a support ticket.
- Beat 14 (admin approval) also carries the module's prerequisite reminder: say it warmly, not as bureaucracy.
- Beat 23: the whiteboard is written on but **the words are added as an `info` overlay**, never baked into the generated art.
- Reuse the four-door grid art from video 1 beat 15 if a recap frame is wanted — do not draw a new one.
- Leave air after beat 8 and after beat 17.
- **No text in any generated scene art.**

Video 3 — The Key Only Opens One Room — draft

Teaches: creating an integration grants your agent nothing — a token is a hotel key card, and in Notion you must hand the card to the room, while in Slack you must add the scope and invite the bot.

One move: share the page, add the scope, reinstall the app, invite the bot — in that order.

Real project / everyday spine: Ali's key is made, his code is correct, and his order table still returns nothing.

This is the video learners will come back to, so it is deliberately **error-first**: both real error strings appear on screen, exactly as they arrive, and each is paired with the single setting that causes it. It teaches no endpoints and no data shapes — video 4 owns those. The hotel key card is introduced here and reused in video 6 for rotation and least privilege.

Deferred: which door to pick (video 2 — assume the learner chose a token here), data sources and IDs (video 4), acknowledgement and duplicates (video 5), where the key is stored and who approves actions (video 6). OAuth for public integrations is out of scope; this is an internal integration.

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	Your agent's key opens only the rooms you hand it to.
2	SCENE	Ali creates his integration and copies the key into his project.
3	SCENE	He runs it, and his order table comes back completely empty. *(act: blank result on the laptop screen, Ali frowning)*
4	INFO	404 · object_not_found *(error card, exact string)*
5	SCENE	He reads his four lines of code four times. The code is fine. *(act: finger tracing the screen)*
6	ALI	Then he reads the message properly: "has not been shared with your integration."
7	INFO	Everybody hits this one. It is not a bug in your code. It is a permission.
8	INFO	A key is a hotel key card, never a master key. *(analogy card: one room, one stay, cancellable)*
9	ALI	The front desk gives you one room, for one stay, and can cancel it any time.
10	SCENE	In Notion you also have to walk the card to the room yourself. *(act: opening the page menu on screen)*
11	INFO	Open the page · the three dots · Connections · add your integration. *(steps card, 4 steps)*
12	SCENE	Ali shares his order table, runs it again, and the rows finally appear. *(act: crisp table filling the screen)*
13	INFO	Parent pages count too. Sharing one child page is not enough. *(gotcha card)*
14	SCENE	Then he tries to post the summary in the shop channel, and Slack refuses. *(act: a red refusal on screen, staff visible through the doorway)*
15	INFO	not_in_channel *(error card, exact string)*
16	ALI	His bot is allowed to write. It is simply not in that room.
17	INFO	Two scopes, then one invite. *(steps card: chat:write · chat:write.public · /invite @yourbot)*

18	INFO	Every new scope needs the app reinstalled, or you get missing_scope.
19	INFO	QUESTION · Your integration returns object_not_found for a database you can see in your own browser. What is wrong? A — the token has expired · B — the database was never shared with the integration · C — your connection dropped · D — you need a paid Notion plan. Write your answer down. *(quiz card, no answer shown, holdAfter)*
20	ALI	Ali hands the key one table and one channel, and nothing else.
21	ALI	Not the whole workspace, because a key that opens everything cannot be trusted with anything.
22	INFO	REVEAL · B. Creating an integration gives it zero access until you share. *(same quiz card, B highlighted)*
23	SCENE	Two errors cost him forty minutes today. Tomorrow they cost him forty seconds. *(act: Ali writing the two error names on a card and pinning it above the desk)*
24	INFO	Share the page. Add the scope. Reinstall. Invite the bot. *(closing checklist card, spoken in full)*
25	ALI	Your turn. Open the one page your agent must read and look at its Connections.
26	ALI	If your integration is not listed there, that is your 404 already waiting.

Gate check

READY · 26 beats · 8 [scene] (31%) · 8 [ali] · 10 [info] · average ~12 words per line

- **One move:** the four-step permission sequence. New concepts: key-as-key-card, sharing a page, parent pages, scopes, reinstall-on-new-scope, channel membership, least privilege. Seven — under the eight ceiling, and every one of them is a setting the learner must actually touch.
- **Leads with the answer:** beat 1 names the rule; the story then earns it twice.
- **Everyday picture:** a hotel key card that has to be walked to the room. Chosen over "the agent joins your team" precisely because a key card *fails closed and silently*, which is what a 404 is.
- **Before → after:** before is an empty table and a red refusal (3–5, 14–15); after is rows on screen (12) and the posted summary (implicit at 20–23). Both errors shown, both fixed on camera.
- **Character:** Ali doubts his code, is told plainly it is not his code, learns the sharing step, hits the second error, fixes that too, ends by pinning the two error names above his desk.
- **Emotional checkpoints:** beat 7 normalises hard ("everybody hits this one, it is not a bug in your code"), beat 16 removes blame from the bot, beat 23 converts the pain into competence. Three.
- **Interactive pair:** QUESTION at beat 19 (~⅔ through, `holdAfter: 6`), REVEAL at beat 22.
- **Exact strings on screen:** `object_not_found`, `not_in_channel`, `missing_scope`, `chat:write`, `chat:write.public`, `/invite @yourbot` — all verified in `research.md` (F4, F5) and reproduced character for character.
- **Numbers:** four lines read four times (5), four steps (11), two scopes plus one invite (17), forty minutes to forty seconds (23).
- **Data templates:** error card x2 (4, 15), key-card analogy card (8), steps card x2 (11, 17), gotcha card (13), quiz card (19, 22), closing checklist (24). Six distinct. Plain statement cards: 7 and 18 — two, at the ceiling.
- **No assignments.** Beat 25 is a spoken, thirty-second check the viewer can do in the browser.

Build notes

- `animateIds = ['03', '10', '12']` — beat 3 (the empty result: the failure that starts everything), beat 10 (walking the key card to the room — the metaphor coming alive, and the single most important action in the module), beat 12 (rows finally appearing: the relief beat). Everything else is a still with a slow pan plus cutout micro-motion.
- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.*
- Beats 4 and 15 are **error cards and must be typo-perfect**; render them in a mono face on the cream card, red only on the error name. Identical geometry for both so the viewer learns the shape.
- Beat 8: draw the key card as a real card in Ali's hand, one room number on it — **the number is an `info` overlay**, not baked art.
- Beats 11 and 17 must **evolve step by step** as the line is spoken, one row lighting at a time. No static four-item list.
- Beat 14: the refusal is crisp HTML on the laptop screen; show two of the three staff through the doorway waiting for a message that never came. That is the cost, and it must be visible.
- Beat 23: the pinned card above the desk becomes a recurring prop in videos 4, 5 and 6 — establish it clearly here.
- Leave air after beat 7 (the normalising beat) and after beat 22.
- **No text in any generated scene art.** Every error string, scope and menu label is an `info` layer.

Video 4 — Rows, Not Paragraphs — draft

Teaches: give your agent a table to write rows into, not a page to write essays on — and target that table by its **data source**, not by the id sitting in your browser address bar.

One move: ask the database what it contains, save the `data_source_id`, and write one row per event.

Real project / everyday spine: Ali's agent writes a beautiful paragraph about the day's orders, and the following week he cannot answer a single question with it.

The concept is "structured output the API can query", and the ID discovery step is the mechanism that makes it work — they are taught as one move, not two. The video shows the *cost* of prose first (a question that cannot be answered) so that named columns feel like relief rather than bureaucracy. The 2025-09-03 database/data-source split is the single most current fact in the module and is stated once, plainly, with the exact discovery call.

Deferred: permissions (video 3 — assume the table is already shared), posting to Slack (video 5), webhooks, secrets and approval (video 6). Relations, rollups and multi-source databases are named at most in passing and never taught.

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	Give your agent rows to fill, not a page to write on.
2	SCENE	Ali's agent writes a beautiful paragraph about today's orders. *(act: prose filling a Notion page on the laptop screen)*
3	ALI	It reads well. It is also completely useless.
4	SCENE	A week later he needs every order above five thousand rupees. *(act: Ali scrolling back through walls of prose, mug forgotten)*
5	INFO	A page holds words. A table holds answers. *(contrast card)*
6	SCENE	So Ali builds one table with five named columns instead. *(act: crisp empty table with headers on screen)*
7	INFO	Date · item · quantity · amount · status *(properties card, five named properties)*
8	SCENE	One row per order, written by the agent, filterable by any of his staff. *(act: rows landing one after another)*
9	SCENE	He copies the id from the browser address bar, queries it, and it fails. *(act: red validation error on screen)*
10	INFO	That id is the box. It is not what is inside the box.
11	INFO	Since the 2025-09-03 version, a database holds one or more data sources. *(concept card: database as container, data source inside)*
12	ALI	You query the data source. The database is only the box around it.
13	INFO	Step one · ask the database what it contains. *(steps card: GET /v1/databases/:id → data_sources[])*

14	INFO	Step two · query the data source you found. *(same card, step two lighting up: POST /v1/data_sources/:id/query)*
15	SCENE	Ali saves that data source id in his config once, and stops guessing forever. *(act: pasting it into a config file on screen)*
16	INFO	QUESTION · You have a database id copied from a Notion URL. What do you do first? A — query that id directly · B — call GET /v1/databases/:id and read its data sources · C — paste it into the page · D — ask an admin for access. Write your answer down. *(quiz card, no answer shown, holdAfter)*
17	INFO	Pin your Notion-Version header, and read the upgrade guide before you change it. *(rule card)*
18	ALI	Versions keep shipping. A pinned version is a project that still works next month.
19	SCENE	The rows are there, and he filters for orders over five thousand in one click. *(act: filtered table, Ali sitting back)*
20	INFO	REVEAL · B. A database id and a data source id are not interchangeable. *(same quiz card, B highlighted)*
21	ALI	One more limit: Notion allows about three requests a second.
22	INFO	Three per second · on a 429, read Retry-After and wait that long. *(limits card)*
23	SCENE	So he writes his sixty rows in a queue, and not one of them fails. *(act: steady rows appearing, wall clock ticking)*
24	INFO	One table. Named columns. One row per event. *(closing card, spoken in full)*
25	ALI	Your turn. Name the five columns your agent's output would fill.
26	ALI	If you cannot name them, your agent is about to write prose again.

Gate check

READY · 26 beats · 8 [scene] (31%) · 8 [ali] · 10 [info] · average ~12 words per line

- **One move:** discover the data source, then write rows. New concepts: table-not-page, named properties, database-as-container, data source, the discovery call, pinned version, three per second. Seven — under the eight ceiling.
- **Leads with the answer:** beat 1, before any story.
- **Everyday picture:** a shopkeeper who wrote his day up in sentences and then could not add up his own takings. The pain is arithmetic, not abstraction.
- **Before → after:** before is prose he has to scroll through (2–4) and a failed query (9); after is a one-click filter (19) and sixty clean rows (23). Same table, same question, visible difference.
- **Character:** Ali is proud of the paragraph, is defeated by his own question a week later, rebuilds as a table, hits the ID trap, learns the two-step discovery, and ends able to answer in one click.
- **Emotional checkpoints:** beat 3 lets him be wrong without shame, beat 10 reframes the failure as a naming problem rather than a coding one, beat 18 reassures that pinning is protection not pedantry.
- **Interactive pair:** QUESTION at beat 16 (~⅔ through, `holdAfter: 6`), REVEAL at beat 20.
- **Exact strings on screen:** `2025-09-03`, `GET /v1/databases/:id`, `data_sources[]`, `POST /v1/data_sources/:id/query`, `Notion-Version`, `Retry-After`, `429` — all verified in `research.md` (F6, F7, F8).

- **Numbers:** five columns (6, 7, 25), five thousand rupees (4, 19), three requests per second (21, 22), sixty rows (23).
- **Data templates:** contrast card (5), properties card (7), concept card (11), two-step evolving steps card (13–14), rule card (17), limits card (22), quiz card (16, 20), closing card (24). Seven distinct. Plain statement cards: 10 and 24 — two, at the ceiling.
- **No assignments.** Beat 25 is a spoken naming exercise, done in the viewer's head.

Build notes

- `animateIds = ['02', '08', '19']` — beat 2 (prose pouring onto the page: the mistake, made attractive on purpose), beat 8 (rows landing one after another — the metaphor of the whole video), beat 19 (the filter snapping to three rows and Ali sitting back: the payoff). Stills plus slow pan everywhere else.
- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.* The pinned error card from video 3 stays above the desk.
- Beats 13–14 are **one evolving card**, not two: step one appears, then step two lights up beneath it with the arrow between them. This card is the takeaway frame of the video — proofread it against `research.md` section D, character by character.
- Beat 11: draw the container relationship literally — one outer box labelled by overlay as the database, one inner card as the data source. Overlay labels only; nothing baked into the art.
- Beat 9: the validation error is crisp HTML on the laptop screen, same card geometry as video 3's error cards so the viewer recognises the shape.
- Beat 6 and beat 19 must be the **same table, same position on screen** — empty then filtered — so the before and after read as one object.
- Beat 23: the queue is shown as steady, evenly spaced rows with the wall clock visible. Never a burst.
- Leave air after beat 5 and after beat 20.
- **No text in any generated scene art.** Column names, endpoints and ids are `info` layers only.

Video 5 — Speaking Without Spamming — draft

Teaches: Slack expects an acknowledgement within three seconds, so your agent must say "coming" first and do the work second — otherwise Slack retries and the same action happens three times.

One move: acknowledge on the first line, then work; deduplicate on the event id.

Real project / everyday spine: Ali's agent answers one question in the shop channel — three times — and then starts replying to itself.

This video owns the duplicate-and-loop failure class, which is the most confusing one for beginners because *nothing in their code looks wrong*. The doorbell is used for exactly one thing — the three-second acknowledgement — and not stretched into a general webhooks lesson. Rate limits and threading are taught as the etiquette that keeps the channel usable, not as trivia.

Deferred: permissions and the invite (video 3), what gets written to Notion (video 4), secrets and human approval (video 6). Block Kit layout, modals, slash-command payload shapes and polling-versus-webhooks theory are all out of scope.

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	Say "coming" in three seconds, then do the work.
2	SCENE	Ali's agent can finally answer questions in the shop channel.
3	SCENE	A staff member asks about last Tuesday, and three identical answers arrive. *(act: three duplicate replies stacking on screen)*
4	ALI	He did not send it three times. Slack asked three times.
5	INFO	Slack expects an OK within three seconds. *(rule card with a three-second clock)*
6	ALI	Slack rings the doorbell. If nobody calls back, it rings again.
7	INFO	No answer in three seconds, then three retries over a few minutes. *(timeline card)*
8	SCENE	His handler was writing the Notion row first, and replying afterwards. *(act: the two lines highlighted on screen in the wrong order)*
9	INFO	Acknowledge first. Do the work after. *(hinge card)*
10	ALI	The fix is not faster code. It is the order of two lines.
11	INFO	Then still check the event id Slack sends, and skip what you have already done. *(card: dedupe on event_id, never on the retry count)*
12	ALI	Same event id, work already finished, so ignore it and carry on.
13	SCENE	One answer arrives now, inside the thread where the question was asked. *(act: a single threaded reply)*
14	INFO	Reply in the thread, not the channel. Attention is not free. *(rule card)*

15	INFO	QUESTION · Your agent posted the same summary three times. What is the most likely cause? A — the network duplicated the message · B — the handler did the work before acknowledging Slack · C — two people asked at the same moment · D — Slack is rate limiting you. Write your answer down. *(quiz card, no answer shown, holdAfter)*
16	SCENE	The next morning, the agent replies to its own message and starts a loop. *(act: messages escalating down the screen)*
17	INFO	Ignore messages that came from your own bot. Always.
18	SCENE	Twenty messages in twenty seconds, and his staff mute the channel. *(act: a phone face-down on the desk, notifications piling up)*
19	INFO	REVEAL · B. Working before you acknowledge means Slack retries while you are still working. *(same quiz card, B highlighted)*
20	INFO	One message per second per channel is the ceiling. *(limits card: 429 · read Retry-After · wait)*
21	SCENE	So the agent posts one line, once, and waits when it is told to wait. *(act: a single calm line on screen, clock visible)*
22	INFO	Building on your own laptop? Use Socket Mode — no public address needed. *(practical card)*
23	SCENE	The channel is quiet again, and one useful line arrives at closing time. *(act: Ali reading it with his mug, staff nodding through the doorway)*
24	INFO	Acknowledge. Deduplicate. Thread. One message. *(closing checklist card, spoken in full)*
25	ALI	Your turn. Find the line in your handler that acknowledges Slack.
26	ALI	If it is not the first thing that runs, your duplicates are already on the way.

Gate check

READY · 26 beats · 8 [scene] (31%) · 7 [ali] · 11 [info] · average ~12 words per line

- **One move:** acknowledge first, then work, keyed on the event id. New concepts: three-second acknowledgement, three retries, duplicates, dedupe on event id, thread replies, ignore your own bot, one message per second, Socket Mode. Eight — at the ceiling, and beats 22 is deliberately the lightest of them (one practical card, no story weight).
- **Leads with the answer:** beat 1 is the whole fix in nine words.
- **Everyday picture:** a doorbell, used for one job only — you must shout "coming" before you walk to the door, or the visitor keeps ringing.
- **Before → after:** before is three identical answers (3) and a twenty-message loop (16–18); after is one threaded reply (13) and one calm closing-time line (21, 23). Both failures are seen, not described.
- **Character:** Ali is embarrassed by his own agent in front of his staff twice, learns that the fault is an ordering mistake rather than bad code, fixes the order, kills the loop, and earns the quiet channel back.
- **Emotional checkpoints:** beat 4 removes the blame immediately ("he did not send it three times"), beat 10 reassures that the fix is small, beat 23 lands the relief. Three.
- **Interactive pair:** QUESTION at beat 15 (~⅔ through, `holdAfter: 6`), REVEAL at beat 19.
- **Exact facts on screen:** three seconds to acknowledge, three retries, `event_id`, one message per second per channel, `429`, `Retry-After`, Socket Mode — all verified in `research.md` (F9, F10, F11).

- **Numbers:** three identical answers (3), three seconds and three retries (5, 7), two lines (10), twenty messages in twenty seconds (18), one per second (20).
- **Data templates:** rule card with clock (5), retry timeline (7), hinge card (9), dedupe card (11), thread rule (14), limits card (20), practical card (22), quiz card (15, 19), closing checklist (24). Eight distinct. Plain statement cards: 9 and 17 — two, at the ceiling.
- **No assignments.** Beat 25 is a spoken one-minute check inside the viewer's own handler.

Build notes

- `animateIds = ['03', '16', '23']` — beat 3 (three duplicate answers stacking: the mystery that opens the video), beat 16 (the self-reply loop escalating — the metaphor and the fear, both alive), beat 23 (the quiet channel and one useful line: the emotional close). Stills plus slow pan elsewhere.
- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.* The pinned error card from video 3 is still above the desk.
- Beat 3 and beat 13 must use the **same channel view, same position** — three stacked replies, then one threaded reply — so the contrast reads instantly.
- Beat 8: highlight the two lines of the handler in the wrong order, then in video-terms *do not* animate the fix here; beat 9's card is the fix. Keep the code shown to two lines. It is crisp screen HTML.
- Beat 7 is an **evolving timeline**: the three-second window, then retry one, two, three appearing along it. Never a static diagram.
- Beat 16–18: the loop must feel physically uncomfortable — accelerate the stacking, then cut to the face-down phone. Do not caption it with a joke; the cost is real.
- Beat 20 shares the geometry of video 4's limits card. Same shape, different numbers, on purpose.
- Beat 22 is a small practical aside; keep it visually quiet so it does not compete with the close.
- Leave air after beat 10 and after beat 19.
- **No text in any generated scene art.** Message bubbles are drawn empty and filled by `info` overlays.

Video 6 — Safe, Then Let It Run — draft

Teaches: before an agent gets real keys, the secret belongs in the environment, a human belongs in front of anything you cannot undo, and every action belongs in a log.

One move: move the key out of the code, name the one action that needs approval, and write down what the agent did.

Real project / everyday spine: Ali pushes his project at eleven at night, proud of it, and wakes up to a dead Slack token.

This is the module close, so it converts the whole series into something Ali can leave running. The leak story is told as a *lucky escape* rather than a disaster, because the honest lesson is that the revocation was the system protecting him. Human-in-the-loop is taught with one word — **before** — and one action, not as a governance framework. The hotel key card from video 3 returns for rotation.

Deferred: nothing. This video may reference every earlier video and closes the module. Secrets managers, audit tooling, staging environments and confidence scoring are named at most once and never taught.

Script (one beat per line · [ali] = character · [scene] = shop scene · [info] = infographic)

1	INFO	Move the secret out of the code, and put a person in front of anything you cannot undo.
2	SCENE	At eleven at night, Ali pushes his project to GitHub, pleased with it. *(act: the push completing on the laptop screen)*
3	SCENE	By morning, his Slack token is dead. *(act: a revoked-token message on screen, Ali's mug untouched)*
4	INFO	The key was in the code. A scanner found it and Slack cancelled it. *(fact card)*
5	ALI	And that was the good outcome. A stranger could have found it first.
6	INFO	The secret lives in a .env file, and .env lives in .gitignore. *(steps card, two lines)*
7	ALI	The code reads the key from the environment, and never holds it.
8	INFO	Then rotate it, because a key you cannot cancel is not a key. *(callback card: the hotel key card from video three)*
9	SCENE	Before testing anything, Ali makes a test channel and a test page. *(act: two new empty spaces on screen)*
10	ALI	Nobody should learn on the real order table, or in the channel everyone reads.
11	INFO	Read-only on day one. One write on day two. *(progression card)*
12	SCENE	His agent may read what it needs, and write in exactly one place. *(act: one table lit, everything else dim)*
13	INFO	QUESTION · Which of these should the agent never do without a human approving first? A — read the orders table · B — post a summary in the test channel · C — delete a customer's page · D — list the channels it can see. Write your answer down. *(quiz card, no answer shown, holdAfter)*
14	SCENE	A week in, the agent proposes deleting a duplicate customer page. *(act: a proposal card waiting on screen, nothing happening yet)*

15	INFO	It proposes. A person approves. Then it happens. *(hinge card, three steps in order)*
16	ALI	The word that matters is "before". Approval afterwards is just news.
17	INFO	Put the human where a mistake is expensive, or cannot be undone. *(rule card)*
18	INFO	REVEAL · C. A deleted page is the one thing Ali cannot get back in a click. *(same quiz card, C highlighted)*
19	SCENE	Everything the agent does gets written down, whether it worked or not. *(act: log lines appearing steadily on screen)*
20	INFO	What it did · what it touched · when · who approved it. *(log card, four fields)*
21	SCENE	When something looks wrong, Ali reads the log instead of guessing. *(act: finger on one log line, expression clearing)*
22	ALI	You cannot fix what you cannot see.
23	SCENE	One table read, one line posted, every action recorded, and the shop closes on time. *(act: calm back room at closing, staff leaving, laptop screen tidy)*
24	INFO	Secret in the environment. Human before the irreversible. A log of everything. *(closing card, spoken in full)*
25	ALI	Your turn. Say out loud the one action your agent must never take alone.
26	ALI	Put a person in front of that one, and you are ready to let it run.

Gate check

READY · 26 beats · 8 [scene] (31%) · 7 [ali] · 11 [info] · average ~12 words per line

- **One move:** secret out, human before the irreversible, log it. New concepts: env file, gitignore, scanners revoke leaked keys, rotation, test channel and test page, read-only first, propose-approve-act, log fields. Eight — at the ceiling, and three of them are single-card practicalities.
- **Leads with the answer:** beat 1 states both halves of the fix.
- **Everyday picture:** a shopkeeper who left a key taped to the shop window, and the locksmith who changed the lock before a stranger tried it.
- **Before → after:** before is a token revoked overnight (2–4) and an agent that could delete a customer page unasked (14); after is a key in the environment (6–7), one write in one place (12), a proposal waiting for a person (15), and a log to read (21). Countable and all on screen.
- **Character:** Ali is proud, then punished mildly, then told he was lucky, then rebuilds carefully, then meets the irreversible action and is *ready* for it, and finally closes the shop on time. The arc of the whole module lands on him, not on a diagram.
- **Emotional checkpoints:** beat 5 reframes the loss as protection, beat 10 makes caution normal rather than timid, beat 22 gives him a sentence to keep, beat 23 pays off the module. Four.
- **Interactive pair:** QUESTION at beat 13 (~½ through by design — the approval discussion needs the answer held across it, `holdAfter: 6`), REVEAL at beat 18.
- **Exact facts on screen:** `.env`, `.gitignore`, leaked keys are detected and revoked by the provider, rotate, propose-approve-act, four log fields — all verified in `research.md` (F13, F14).
- **Numbers:** eleven at night to morning (2–3), one test channel and one test page (9), day one and day two (11), one write location (12), four log fields (20).

- **Data templates:** fact card (4), steps card (6), key-card callback (8), progression card (11), hinge three-step card (15), rule card (17), log card (20), quiz card (13, 18), closing card (24). Eight distinct. Plain statement cards: 17 and 24 — two, at the ceiling.
- **No assignments.** Beat 25 is spoken aloud by the viewer; the module's graded work lives in the separate assessment, never in the video body.

Build notes

- `animateIds = ['03', '15', '23']` — beat 3 (the dead token: the shock that opens the video), beat 15 (propose, approve, act — the module's most important mechanism, and it must be seen as three steps in time), beat 23 (the shop closing on time: the emotional close of the whole module). Stills plus slow pan everywhere else.
- **Setting const** in every `scene` prompt: *the back room of Ali's stationery shop — cream walls, a wooden desk, a laptop with a plain blank screen, a fat paper order file, a chipped blue mug, a wall clock, boxes of notebooks stacked behind.* The pinned error card from video 3 is still above the desk; by beat 23 the paper order file is closed and pushed aside — the module's silent before-and-after.
- Beat 2 is the only night-lit frame besides video 2 beat 16; match that lighting exactly.
- Beat 8 **reuses video 3's key-card art** with the room number swapped, so rotation reads as "new card, same room". Do not draw a new key.
- Beat 15 is the hinge frame and must **evolve in three stages** as the line is spoken: proposal appears, a hand approves, the action completes. If any single frame in this module gets extra care, it is this one.
- Beat 14: the proposal card sits there **doing nothing** for the whole beat. The stillness is the point; resist adding motion beyond a slow pan.
- Beat 20's four fields appear one at a time, matching the four spoken phrases.
- Beat 23 is the module's last shop frame: same angle as video 1 beat 2, but tidy, lit, and empty of paper. Shoot it as the deliberate bookend.
- Leave air after beat 5, after beat 16, and after beat 22.
- **No text in any generated scene art.** File names, log fields and the revoked-token message are all `info` overlays.

Video 7 — Notion, Click by Click — draft

Format: SCREENCAST (not the Ali illustration format). Every step is a real Notion / editor screen, rendered the way the Module 11 assessment guide is rendered — `card` + `ui` beats only, no Imagen art, TTS-only cost. Ali appears in voice at the open and close so the series still holds together.

Teaches: the exact seven steps — across ten screens — that take a Notion integration from "created" to "returning my rows".

One move: create it, share the page under Connections, then discover the data source id.

Real project / everyday spine: we are wiring the key for Ali's order agent, and the viewer follows along in their own workspace.

This is the video learners will pause and replay, so it is built for **following, not for watching**: one action per beat, the cursor visible, and every field named out loud before it is typed. It teaches nothing new conceptually — videos 3 and 4 already earned the ideas. Its only job is that the viewer's own integration returns rows by the end.

Deferred: Slack (video 8), everything conceptual (videos 1–6). Public OAuth integrations, webhooks and multi-source databases are out of scope.

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [ali] = character)

1	CARD	Five minutes, seven steps, and your agent can read your Notion table. *(title card)*
2	ALI	We are making the key for Ali's order agent — do it in your own workspace as we go.
3	UI	Go to notion dot so slash my dash integrations, and click New integration. *(N1 · cursor to the button)*
4	UI	Name it after the job, choose your workspace, and leave the type as internal. *(N1 · form filling in)*
5	UI	Give it three capabilities — read content, update content, insert content. *(N3 · three checkboxes ticking one at a time)*
6	UI	Nothing more than that. A key should open as little as possible. *(N3 · the unticked boxes held)*
7	UI	Copy the secret once, and paste it straight into your dot env file. *(N2 → N2b · secret revealed then landing in .env)*
8	UI	Not into your code. A scanner will find it there, and your key will be cancelled. *(N2b · .gitignore visible below)*
9	CARD	That is the easy half. The half everyone skips is next. *(signpost card)*
10	UI	Open the page you want it to read, and click the three dots, top right. *(N4 · cursor to the dots, menu opening)*
11	UI	Scroll down to Connections, and click it. *(N4 · menu scrolling, Connections highlighted)*
12	UI	Search your integration by name, and confirm. *(N5 · name typed, result selected)*
13	UI	Read that confirmation. This is the exact moment access is granted. *(N5 · the confirm dialog held)*

14	UI	If your table sits inside another page, do this on the parent page too. *(N4b · a nested page tree, parent highlighted)*
15	CARD	QUESTION · You added your integration under Connections on the child page, and you still get object_not_found. What is the most likely cause? A — the secret is wrong · B — the parent page was never shared · C — Notion is down · D — the integration needs reinstalling. Write your answer down. *(quiz card, no answer shown, holdAfter)*
16	UI	Now the id. It is in the address bar, and it is the part before the question mark. *(N6 · address bar, the id segment highlighted)*
17	UI	The part after v equals is the view. That is the one beginners copy by mistake. *(N6 · the ?v= segment highlighted in a different colour)*
18	UI	Ask the database what it holds, and read the data source id out of the reply. *(N7 · the GET call and the data_sources array on screen)*
19	UI	Then query that data source, and your rows come back. *(N7b · rows printing in the terminal)*
20	CARD	REVEAL · B. Sharing the child page is not enough — the parent has to be shared too. *(same quiz card, B highlighted)*
21	CARD	Create · capabilities · secret · Connections · confirm · id · first rows. *(checklist card, the seven steps, ticking in sequence)*
22	ALI	Ali's agent can read the order table now, and nothing else in the workspace.
23	ALI	Your turn. Do those seven steps, then come back for the Slack half.

Screens to capture or re-create — ten screens across the seven steps

ID	Screen	Must be visible	Must NOT be visible
N1	`notion.so/my-integrations` → **New integration** form	The "New integration" button, then name field, workspace picker, type = Internal	Other integrations in the list, the real workspace name
N2	Integration created → **Internal Integration Secret**	The Show / Copy control, the secret masked as dots	Any real secret characters, even blurred
N2b	Editor: `.env` and `.gitignore` side by side	`NOTION_TOKEN=` with a masked value, `.env` listed inside `.gitignore`	A real token
N3	Integration **Capabilities** section	Read content, Update content, Insert content ticking one at a time; user-information options left off	—
N4	The order database page → `...` menu open → **Connections**	The three-dot button, the open menu, Connections highlighted	Real customer or order data — use invented rows
N4b	Same page inside a **parent** page	The sidebar tree showing parent → child, parent highlighted	Real page titles from the workspace
N5	**Connections** → search → confirm dialog	The typed integration name, the result row, the confirmation wording about access	—
N6	Browser address bar of the database page	The 32-character id segment highlighted; the `?v=…` view id highlighted separately	The real workspace subdomain

ID	Screen	Must be visible	Must NOT be visible
N7	Terminal / editor: `GET /v1/databases/:id` response	`Notion-Version` header, the `data_sources` array, one `id` inside it	A real token in the header — mask it
N7b	Terminal: `POST /v1/data_sources/:id/query` response	Three or four invented order rows printing	—

Two production routes. *(recommended: re-create)*

- **Re-create in HTML** — a new `animation/walkthrough.html` built on `assessment.html`'s chrome, with a browser frame instead of the IDE window. Costs nothing, is proofread-able, re-renders in seconds when a vendor moves a button, and keeps the house rule that no text is ever baked into art. Use plain text wordmarks, no vendor logos, and invented shop data throughout.
- **Real screengrabs** — Aroma captures the ten screens from her own workspace. More authentic, but every frame needs a pass for real names, real page titles and real token characters, and it goes stale silently when the UI changes. Worth it for N4 and N5 only, if we want two hero frames.

Gate check

READY · 23 beats · 15 [ui] (65%) · 5 [card] · 3 [ali] · average ~13 words per line

- **Format exception, deliberate:** the evals-grade visual standard (28% `scene`, illustrated setting, real-laptop art) does not apply to the screencast format — the same exception the Module 11 assessment guide runs under. The visual standard here is: one action per screen, a visible cursor, and every element named before it is clicked. Flagging it so the visuals gate is not run against this file.
- **One move:** create → share under Connections → discover the data source. New concepts: none. Every idea here was earned in videos 3 and 4; this is execution only.
- **Leads with the answer:** beat 1 states the whole job and its length.
- **Before → after:** before is a created integration that returns nothing; after is rows printing in a terminal (19). The viewer's own workspace is the before-and-after.
- **Emotional checkpoints:** beat 2 invites them to follow along at their own pace, beat 9 warns them that the skipped step is coming (rather than letting them fail), beat 17 names the mistake as normal. Three.
- **Interactive pair:** QUESTION at beat 15 (~⅔ through, `holdAfter: 6`), REVEAL at beat 20.
- **Exact strings on screen:** `object_not_found`, `Notion-Version`, `data_sources`, `GET /v1/databases/:id`, `POST /v1/data_sources/:id/query`, `.env`, `.gitignore` — all verified in `research.md` (F4, F6, F13).
- **Numbers:** seven steps (1, 21) shot across ten screens, three capabilities (5), five minutes (1). The spoken count is always *steps*; the capture table counts *screens*.
- **No assignments.** Beat 23 is a spoken hand-off to video 8.

Build notes

- **No Imagen art, no i2v, no `animateIds`.** Movement comes from the renderer: cursor travel, click pulses, typed text appearing character by character, and menus scrolling. Every beat must contain one of those — a still interface screen is a failed beat in this format.
- Add a **step counter** in the corner (`3 / 7`) that advances on beats 3, 5, 7, 10, 12, 16, 18 — this is a follow-along video and people need to know where they are.

- `holdAfter` on beat 15 as usual; also **hold beats 13 and 16 longer than the voiceover** — those are the two frames viewers will pause on.
- Beats 10–13 are the heart of the video. One continuous browser frame, no cuts, cursor moving between them, so the whole Connections path reads as a single motion.
- Beat 17 must **contrast the two ids in one frame** — same address bar, two different highlight colours, the wrong one clearly marked. Do not show it as two separate screens.
- Beat 21's checklist ticks in the same order the video performed the steps, and its wording matches the step counter labels exactly.
- Ali's two beats (2, 22–23) use the standard illustrated shop frame from the concept videos, so the series bookends visually. They are the only illustrated frames in this video.
- Every masked value must be masked **in the render**, not blurred afterwards.

Video 8 — Slack, Click by Click — draft

Format: SCREENCAST (not the Ali illustration format). Same treatment as video 7 — `card` + `ui` beats only, real interface screens, no Imagen art, TTS-only cost. Ali appears in voice at the open and close.

Teaches: the exact eight steps — across fifteen screens — that take a Slack app from "created" to "posting in one channel".

One move: add the scope, reinstall, copy the bot token, invite the bot to the channel.

Real project / everyday spine: the second half of Ali's order agent — giving it a voice in one shop channel, and only that one.

Built for following, not watching. The video's spine is the pattern it shares with video 7:

permission, then membership — a scope is not access, and neither is an installed app. That sentence is the reason these two walkthroughs are a pair rather than one long video.

Deferred: Notion (video 7), everything conceptual (videos 1–6). Block Kit layout, slash-command payloads, modals and distributing an app to other workspaces are out of scope.

Script (one beat per line · [card] = full-screen card · [ui] = interface screen · [ali] = character)

1	CARD	Eight steps, and your agent can post in one channel — only that one. *(title card)*
2	ALI	Same project, other half. Now Ali's agent gets a voice in the shop channel.
3	UI	Go to a p i dot slack dot com slash apps, and click Create New App. *(S1 · cursor to the button)*
4	UI	Choose From scratch, name it after the job, and pick your workspace. *(S1b · the modal filling in)*
5	UI	If your workspace needs an admin to approve apps, ask them now, not on Friday. *(S1c · the approval notice held)*
6	UI	Open OAuth and Permissions, and scroll down to Bot Token Scopes. *(S2 · sidebar click, page scrolling)*
7	UI	Add chat colon write. That is permission to speak at all. *(S2 · scope typed and added)*
8	UI	Add chat colon write dot public only if it must post in channels it has not joined. *(S2b · second scope added)*
9	UI	Every new scope makes Slack ask you to reinstall. Do it, or you get missing scope. *(S2c · the yellow reinstall banner)*
10	UI	Click Install to Workspace, and actually read what you are agreeing to. *(S3 · the consent screen held)*
11	UI	Copy the Bot User OAuth Token — the one beginning x o x b. *(S4 · masked token, Copy clicked)*
12	UI	It goes into dot env, beside your Notion key, and dot env stays in dot gitignore. *(S5 · both files on screen)*

13	CARD	QUESTION · Your bot has chat colon write and a valid token, and posting to your private test channel returns not_in_channel. What is missing? A — another scope · B — the bot is not a member of that channel · C — a paid Slack plan · D — the app was never installed. Write your answer down. *(quiz card, no answer shown, holdAfter)*
14	UI	Make a test channel first. Not general, not the channel your team actually reads. *(S6 · new channel dialog, an invented test name)*
15	UI	Type slash invite, then your bot's name, and send it. *(S6b · the composer, typed character by character)*
16	UI	Slack confirms it joined. That is the step people miss. *(S6c · the joined confirmation in the channel)*
17	CARD	It is the same pattern as Notion — permission, then membership. *(hinge card, two columns: scope / invite · shared / connected)*
18	UI	Now post one line from your code, and watch it arrive with the APP badge. *(S7 · the first bot message appearing)*
19	CARD	REVEAL · B. The scope lets it speak; membership lets it speak *here*. *(same quiz card, B highlighted)*
20	UI	Working on your own laptop with no public address? Turn on Socket Mode. *(S8 · the Socket Mode toggle, then the app-level token beginning x a p p)*
21	UI	And put that first message in a thread, not straight into the channel. *(S7b · the same message, threaded)*
22	CARD	Create · scopes · reinstall · install · token · env · invite · first message. *(checklist card, the eight steps, ticking in sequence)*
23	ALI	Ali's agent now reads one table and speaks in one channel.
24	ALI	That is the whole integration. Everything after this is what you choose to do with it.

Screens to capture or re-create — fifteen screens across the eight steps

ID	Screen	Must be visible	Must NOT be visible
S1	`api.slack.com/apps` → **Create New App**	The Create New App button	Other apps in the list, the real workspace name
S1b	**From scratch** modal	App name field filled with the job name, workspace picker	The real workspace name
S1c	Admin-approval notice	The wording that an admin must approve the install	—
S2	**OAuth & Permissions** → Bot Token Scopes	`chat:write` being added from the scope picker	Any user-token scopes — do not show that section at all
S2b	Same, second scope	`chat:write.public` added beneath it	—
S2c	The **reinstall** banner	The yellow "you changed permission scopes, reinstall your app" strip	—
S3	**Install to Workspace** consent screen	The list of what the app will be able to do, the Allow button	The real workspace name and icon
S4	**Bot User OAuth Token**	`xoxb-` prefix then dots, the Copy control	Any real token characters, even partially

ID	Screen	Must be visible	Must NOT be visible
S5	Editor: `.env` and `.gitignore`	`SLACK_BOT_TOKEN=` masked, `NOTION_TOKEN=` masked above it, `.env` inside `.gitignore`	Real tokens
S6	New channel dialog	An invented test channel name, set to private	Real channel names from the sidebar
S6b	Channel composer	`/invite @order-agent` typed out character by character	Real colleagues' names or avatars in the member list
S6c	Channel after the invite	The "added to the channel" confirmation line	—
S7	The bot's first message	The message with the **APP** badge next to the bot name	Real message history above it — use invented lines
S7b	The same message, threaded	The thread reply count / thread view	—
S8	**Socket Mode** toggle + App-Level Token	The toggle switched on, `xapp-` prefix then dots	Any real token characters

Production route: same as video 7 — recommend re-creating these in `animation/walkthrough.html` (plain text wordmarks, no vendor logos, invented shop data, values masked in the render itself). Real captures are only worth it for S3 and S6c, and every real frame needs a pass for colleague names, channel names and token characters.

Gate check

READY · 24 beats · 16 [ui] (67%) · 5 [card] · 3 [ali] · average ~13 words per line

- **Format exception, deliberate:** as with video 7, the evals-grade visual standard does not apply to the screencast format — flagging it so the visuals gate is not run against this file. The bar here is one action per screen, a visible cursor, and every element named before it is clicked.
- **One move:** scope → reinstall → token → invite. New concepts: none conceptually; the only new *facts* are the two token prefixes (`xoxb-`, `xapp-`) and where each lives.
- **Leads with the answer:** beat 1 states the job, the length and the blast radius ("only that one").
- **Before → after:** before is an app that exists and cannot speak; after is a threaded message with the APP badge (18, 21) in the viewer's own test channel.
- **Emotional checkpoints:** beat 5 stops them being blocked by an admin on Friday, beat 16 tells them the missed step is normal rather than careless, beat 24 hands the project back to them. Three.
- **Interactive pair:** QUESTION at beat 13 (~½ through — placed early on purpose so the invite steps play *while* the answer is still open, `holdAfter: 6`), REVEAL at beat 19.
- **Exact strings on screen:** `chat:write`, `chat:write.public`, `missing_scope`, `not_in_channel`, `/invite`, `xoxb-`, `xapp-`, Socket Mode, `.env`, `.gitignore` — all verified in `research.md` (F5, F11, F13).
- **Numbers:** eight steps (1, 22) shot across fifteen screens, two scopes (7, 8), one channel (1, 23). The spoken count is always *steps*; the capture table counts *screens*.
- **No assignments.** Beat 24 closes the module's practical half.

Build notes

- **No Imagen art, no i2v, no `animateIds`.** Movement is cursor travel, click pulses, typed text and the reinstall banner sliding in. A still interface screen is a failed beat.
- **Step counter** in the corner (`4 / 8`), advancing on beats 3, 6, 9, 10, 11, 12, 15, 18.
- Beat 17 is the hinge frame of both walkthroughs: a two-column card, Notion on the left (shared → connected), Slack on the right (scope → invited), with the shared word **membership** landing last. It is the frame that justifies videos 7 and 8 being a pair. Give it the most care in this file.
- Beats 14–16 run as one continuous Slack frame, cursor moving, no cuts — the invite is the step that fails silently for people, so it must be seen happening in one motion.
- Beat 9's banner is the only yellow element in the video. Do not decorate anything else in that colour.
- Beat 20 is a quiet aside for local development; keep it visually calm so it does not compete with the close.
- Beat 22's checklist wording matches the step counter labels exactly, and ticks in performed order.
- Ali's beats (2, 23–24) use the illustrated shop frames; beat 23 should reuse video 6's closing composition so the module lands on the same picture it ended on.
- Every masked value is masked **in the render**, never blurred afterwards.

Module 14 — Research Brief: connecting an agentic project to Notion and Slack

Scope: what a **beginner** must actually do, in order, for a Notion + Slack integration to go smoothly — and the specific errors they will hit if they skip a step. Six videos.

A. Verified facts to teach (each with source + date)

#	Fact	Source	Date
F1	There are two ways in, not one: a **hosted MCP connector** (OAuth, no tokens to manage, tool schemas maintained by the vendor) or an **API token** (your own code, headless, event-driven). MCP is "the right default for nearly everyone" working *interactively* in a chat/IDE client; the API is required for anything that must run unattended or react to events, because MCP has no event-subscription surface.	scalekit.com — Notion MCP vs Notion API · StackOne	2026
F2	Notion's hosted MCP server is `https://mcp.notion.com/mcp`, OAuth-only, added in Claude Code with `claude mcp add --transport http notion https://mcp.notion.com/mcp` then `/mcp` to authorise.	developers.notion.com — Connect to Notion MCP · Notion blog	2026
F3	Slack ships an **official Slack-hosted MCP server** at `https://mcp.slack.com/mcp`, GA **17 Feb 2026**, user-token OAuth, inherits the authenticating user's permissions and **requires workspace-admin approval**. It replaced Anthropic's reference implementation (archived May 2025).	docs.slack.dev — Connect to Claude · usecarly.com	2026-02-17
F4	**Creating a Notion integration grants it zero access.** You must open each page/database, then `...` → **Connections** → add the integration. Parent pages must be shared too — sharing only a child is not enough. Skipping this returns HTTP **404** `object_not_found`: "The page, database, or block either does not exist or has not been shared with your integration." This is the single most common Notion API error.	makenotion/notion-mcp-server #81 · SFAI Labs — Notion API key setup	2026
F5	Slack bot token (`xoxb-`) lives under **OAuth & Permissions**; posting needs `chat:write` (plus `chat:write.public` to post in public channels it has not joined). **Every new scope requires reinstalling the app.** Without `chat:write.public`, or in a private channel, posting fails with `not_in_channel` until you run `/invite @yourbot`.	docs.slack.dev — Scopes · Prismatic Slack component docs	2026
F6	Notion API version **2025-09-03** split databases from **data sources**: a database is now a *container* of one or more data sources. `POST /v1/databases/:id/query` becomes `POST /v1/data_sources/:id/query`; retrieving/updating a schema and creating pages also move to `data_source_id`. **Database IDs and data source IDs are not interchangeable.** The required discovery step: call `GET /v1/databases/:database_id` with `Notion-Version: 2025-09-03` and read the returned `data_sources` array, then persist that `data_source_id`.	developers.notion.com — Upgrade guide 2025-09-03 · Upgrade FAQs	2025-09-03

#	Fact	Source	Date
F7	The `Notion-Version` header is **required** and versions are named for their release date; Notion has continued shipping versions through 2026 (2026-02-01, 2026-03-01, 2026-04-01 among them). Teachable rule: **pin a version you have tested and read the upgrade guide before bumping it** — do not teach "the latest version is X", it moves.	developers.notion.com — Versioning	2026
F8	Notion enforces an average of **3 requests per second per integration** (~2,700 per 15 min), on all plans, with no paid tier for more. Handle **429** by reading the `Retry-After` header (integer seconds) and pausing at least that long; queue and serialise calls rather than bursting.	developers.notion.com — Request limits	2026
F9	Slack allows **no more than one message per second per channel** (whether via `chat.postMessage`, an incoming webhook, or anything else); short bursts tolerated. Web API methods sit in tiers (Tier 1 = 1+/min … Tier 4 = 100+/min) per method, per workspace, per app. Sustained overrun returns **HTTP 429** `rate_limited` with a `Retry-After` header.	docs.slack.dev — Rate limits	2026
F10	Slack Events API: your endpoint must return **HTTP 200 within 3 seconds**. If it does not, Slack **retries three times** over a few minutes — so if you do the real work (write to Notion, post a message) *before* acknowledging, you get **duplicate actions**. Fix: acknowledge first, then work; deduplicate on the stable `event_id`, not on the retry counter.	docs.slack.dev — The Events API · Question Base writeup	2026
F11	**Socket Mode** delivers events over a persistent WebSocket instead of a public HTTP endpoint — the correct choice for local development, machines behind a firewall, or anywhere you cannot expose a URL.	docs.slack.dev — The Events API	2026
F12	Notion webhooks: on creating a subscription Notion POSTs a one-time `verification_token` to your URL, which you paste back into the integration's Webhooks tab to activate — and which then doubles as the **signing secret**. Every delivery carries `X-Notion-Signature` (HMAC-SHA256 of the body). Lose the token and you must delete and recreate the subscription.	Hookdeck — securing Notion webhooks · developers.notion.com — Webhooks	2026
F13	A token committed to a public GitHub repo is detected by **secret scanning** (200+ token formats, 100+ providers, including Slack tokens and webhooks) and **reported to the provider, who may revoke it immediately** — often before the developer sees the alert. Slack also proactively hunts for its own leaked tokens and disables them. Practice: env vars or a secrets manager, least privilege, rotate.	GitHub Changelog — secret scanning validation for Slack · GitGuardian — remediating Slack bot token leaks	2024-04-11 / 2026
F14	Human-in-the-loop means the agent **proposes and a human approves before the action happens** — "the key word is before". Put the human where an error is expensive, irreversible, legally significant or hard to detect; let the agent read freely, watch it on a dashboard, and stop it before the irreversible thing. Log the proposed action, approver, timestamp, edits and final state.	Velt — AI agent approval layer · Strata — 2026 guide to HITL	2026

B. Strongest analogies found

USE — the hotel key card (for tokens + scopes). "The front desk doesn't give you a master key. They give you a keycard that opens only your room — and only for the length of your stay. Secure. Limited. Revocable." Scopes are which doors the card opens. (Okta, 2019 — the analogy is evergreen.) It carries the

Notion twist perfectly: in Notion you also have to **hand the card to the room** (share the page), or the card opens nothing.

USE — the doorbell vs the mailbox (for events vs polling). "Polling is checking your mailbox every hour for a package. A webhook is the delivery person ringing your doorbell when it arrives." (Merge, 2026.)

AVOID — "the agent joins your team like a new colleague." Misleads on permissions: a colleague can ask someone for access, an integration silently gets a 404 and stops. It also feeds the "it just knows our workspace" misconception (M1 below).

AVOID — plumbing / "wiring it up". Beginners already believe the hard part is code. The hard part is **permissions**, and pipe metaphors hide that.

C. Top learner misconceptions → these become the failure-mode beats

- **M1. "I made the integration / installed the app, so it can see my workspace."** No. Notion access is granted page by page (F4) and Slack posting needs both a scope and channel membership (F5). This is the number-one beginner wall and owns **video 3**.
- **M2. "An ID from the browser URL is the ID the API wants."** Since 2025-09-03 the thing you query is a **data source**, discovered from the database (F6). Owns **video 4**.
- **M3. "My handler can do the work and then reply to Slack."** Three seconds, three retries, three copies of the same action (F10) — plus the bot that replies to its own message and loops. Owns **video 5**.
- **M4. "I should build with the API because that's the real way."** For interactive work the connector is faster and safer; the API earns its keep when the job runs unattended or reacts to events (F1). Owns **video 2**.
- **M5. "Secrets in the code are fine, it's a private repo."** F13. Owns **video 6**.

D. Current names / numbers safe to put on an `info` card

`https://mcp.notion.com/mcp` · `https://mcp.slack.com/mcp` (GA 17 Feb 2026) · `claude mcp add --transport http notion https://mcp.notion.com/mcp` · `xoxb-` bot token · `chat:write`, `chat:write.public` · `/invite @yourbot` · 404 `object_not_found` · `not_in_channel` · `missing_scope` · `Notion-Version: 2025-09-03` · `GET /v1/databases/:id` → `data_sources[]` → `POST /v1/data_sources/:id/query` · Notion **3 requests/second** · Slack **1 message/second/channel** ·

3 seconds to acknowledge, **3** retries, dedupe on `event_id` · 429 + `Retry-After` · `.env` + `.gitignore` · `X-Notion-Signature`

Do not put on a card: "the latest Notion API version is …" (F7 — it moves). Say *pin your version*.

E. Best teaching technique for this topic

Error-first, contrast pair. For integration work the durable teaching unit is *"here is the exact error message → here is the one setting that causes it"*. Beginners do not fail at concepts, they fail at four or five specific 404s and permission errors, and they cannot tell a permission message from a code bug. So each video stages the real error on screen, names it, and hands over the single fix — paired against the working version (before →

after). Secondary technique: **one worked example run in depth** (one Notion table, one Slack channel, one action) rather than a survey of possibilities.

Ordering follows the actual smooth path: *decide what crosses the boundary (1) → pick the door (2) → get permission right (3) → write structured data (4) → speak without spamming (5) → make it safe (6)*.